RAGGit

Single-tenant, on-prem RAG system

Ανδρέας Δασκαλόπουλος

Patras Tech Talk · 26 Νοεμβρίου 2026 · Πάτρα

Τι σημαίνει «ανεβάζω ένα έγγραφο σε AI»;

- Το αρχείο και η ερώτηση φεύγουν από τον υπολογιστή και φτάνουν στην υπηρεσία του παρόχου.
- Ιδιαίτερα για τα αρχεία, αυτά αποθηκεύονται σε vector stores.
- Η πολιτική εξαρτάται από την υπηρεσία, το πλάνο και τις ρυθμίσεις: εκπαίδευση, διατήρηση, συνεργάτες και συνδεδεμένα εργαλεία.
- Παραμένουν και οι κίνδυνοι παραβίασης ή νόμιμου αιτήματος πρόσβασης.

Παράδειγμα πολιτικών υπηρεσιών AI

- OpenAI: στους προσωπικούς λογαριασμούς Free, Plus και Pro, η κοινή χρήση για βελτίωση είναι ενεργή από προεπιλογή.
- Anthropic: η χρήση συνομιλιών για βελτίωση εξαρτάται από συναίνεση και ορισμένες εξαιρέσεις ασφάλειας.
- xAI: υπάρχουν ρυθμίσεις εκπαίδευσης. Το Private Chat δεν χρησιμοποιείται για εκπαίδευση και διαγράφεται εντός 30 ημερών.

Τι θέλουμε να πετύχουμε

Στόχος

Ένα RAG σύστημα αποκλειστικά για ένα οργανισμό (single-tenant) όπου τα LLM, τα RAG pipelines και τα δεδομένα μας είναι όλα τοπικά (on premises).

Προς

- Υπουργεία, δήμοι, νοσοκομεία, ιατρεία, φορολογικά και δικηγορικά γραφεία.
- Μεγάλος όγκος ψηφιακών αρχείων.
- Έγγραφα, ερωτήματα και απαντήσεις που πρέπει να παραμένουν ιδιωτικά.
- Διατεθειμένοι να καλύψουν το κόστος εξοπλισμού για τα τοπικά AI models.

Retrieval-Augmented Generation (RAG)

Ορισμός

Μια μέθοδος που δίνει στα μεγάλα γλωσσικά μοντέλα (LLMs) τη δυνατότητα να ψάχνουν και να βρίσκουν στοιχεία σε εξωτερικές πηγές δεδομένων, προτού διαμορφώσουν την απάντησή τους.

RAG pipelines

- Data Ingestion (Εισαγωγή των δεδομένων)
- Retrieval & Generation (Ερώτηση και Απάντηση)

Data Ingestion

1. **Εξαγωγή Text**
   Αφαίρεση περιττών κενών, κεφαλίδων, υποσέλιδων και αριθμών σελίδων.
2. **Τεμαχισμός (Chunking)**
   Τα μοντέλα διαβάζουν περιορισμένο κείμενο τη φορά, οπότε το σύστημα κόβει τα μεγάλα έγγραφα σε κομμάτια (chunks).
3. **Μετατροπή σε Διανύσματα (Embeddings)**
   Ένα ειδικό μοντέλο (Embedding LLM) μετατρέπει κάθε κομμάτι σε λίστα αριθμών (διάνυσμα) που αποτυπώνει το νόημά του.
4. **Αποθήκευση στη Διανυσματική Βάση (Vector Database)**
   Τα διανύσματα, με το αρχικό κείμενο και τα μεταδεδομένα (τίτλος, σελίδα), αποθηκεύονται σε Vector Database.

Retrieval & Generation

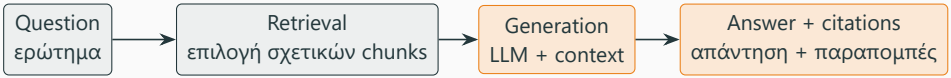
1. **Ανάκτηση (Retrieval)**. Όταν κάνετε μια ερώτηση, το σύστημα τη μετατρέπει και αυτήν σε αριθμούς (με το ίδιο Embedding LLM). Ψάχνει στη βάση δεδομένων και εντοπίζει τα κομμάτια κειμένου που ταιριάζουν μαθηματικά περισσότερο με το νόημα της ερώτησής σας.
2. **Επαύξηση (Augmentation)**. Το σύστημα παίρνει την ερώτησή σας, κολλάει από κάτω τα σχετικά αποσπάσματα που μόλις βρήκε και τα στέλνει μαζί στο Query LLM, το γλωσσικό μοντέλο που συντάσσει την απάντηση.
3. **Παραγωγή (Generation)**. Το Query LLM διαβάζει τα συγκεκριμένα αποσπάσματα και συντάσσει μια φυσική, τεκμηριωμένη απάντηση, με λιγότερες επινοήσεις (hallucinations).

RAG pipelines

Data Ingestion · μία φορά ανά έγγραφο

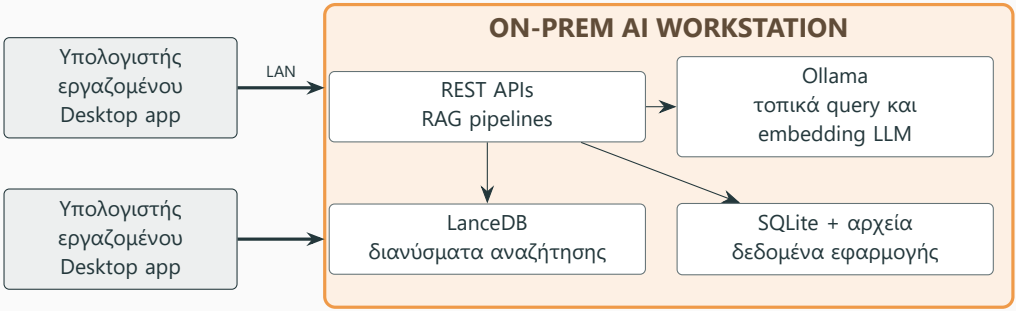


Query · σε κάθε ερώτηση



Η απάντηση βασίζεται στα chunks που επέστρεψε το retrieval και περιλαμβάνει citations προς τις πηγές.

Workstation + Desktop clients



LLM: τοπικά μοντέλα μέσω Ollama

Embedding LLM

- BGE-M3 (1024 διαστάσεις)
- Μετατρέπει chunks και ερωτήσεις σε διανύσματα
- Το ίδιο μοντέλο και στις δύο φάσεις

Query LLM

- Qwen3 4B Instruct
- Απαντά μόνο με βάση τα αποσπάσματα

Τα μοντέλα είναι **configurable** (στους `appsettings` του workstation): σε ισχυρότερο hardware επιλέγεται μεγαλύτερο Query LLM, χωρίς αλλαγή στον κώδικα. Τα δικά μας είναι μικρά, γιατί η ανάπτυξη έγινε σε έναν Intel NUC «Skull Canyon» — οι επιλογές περιορίστηκαν από το υλικό που είχαμε διαθέσιμο.

Όλα εκτελούνται τοπικά μέσω Ollama — καμία κλήση σε εξωτερική υπηρεσία.

Βάσεις δεδομένων

LanceDB — διανύσματα

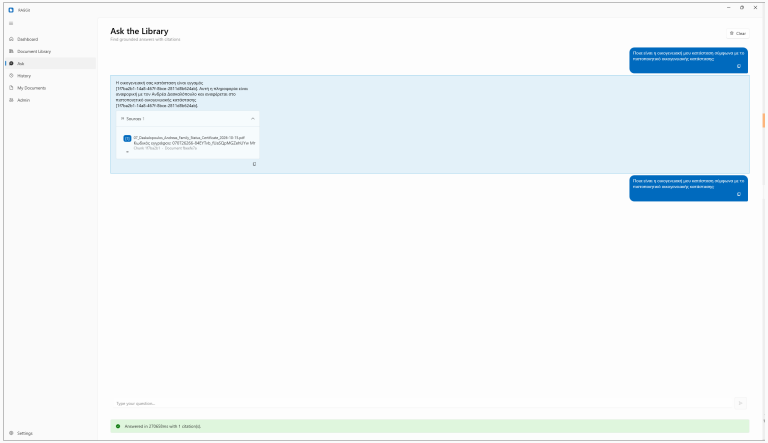
- Embeddings + κείμενο chunks + metadata
- ANN αναζήτηση (cosine, HNSW index)
- Τοπική αποθήκευση σε αρχεία στον δίσκο

SQLite — δεδομένα εφαρμογής

- Έγγραφα, καταστάσεις ingest, χρήστες
- Chunks (παιδιά/γονείς) και ιστορικό ερωτήσεων
- Ένα αρχείο `rag.db`, χωρίς server

Τα πρωτότυπα αρχεία των εγγράφων φυλάσσονται στο ίδιο workstation (ένα αρχείο ανά document id).

RAGGit desktop app



Technology stack

Εφαρμογή

- C# / .NET 10 — ASP.NET Core Web API
- WPF + WPF-UI (Fluent, MVVM) client
- PdfPig: εξαγωγή κειμένου από PDF
- JWT αυθεντικοποίηση με ρόλους

Δοκιμές και παράδοση

- TDD: 700+ τεστ σε τέσσερα επίπεδα
- GitHub Actions CI σε κάθε αλλαγή
- Spec-driven ανάπτυξη ανά feature
- Docker Compose με TLS

Ο κώδικας είναι ανοιχτός: github.com/daskas-welt/raggit

Τι διαφοροποιεί το RAGGit

Single-tenant, on-prem

- Μια εγκατάσταση ανά οργανισμό
- Δεδομένα και μοντέλα στον χώρο του πελάτη
- Έλεγχος πρόσβασης και αντιγράφων ασφαλείας

Συντονισμός στον πελάτη

- Query / embedding LLM — επιλογή με μέτρηση
- Chunk size και TopK ανά βιβλιοθήκη
- Δοκιμή στο δικό του υλικό
- Όλα ρυθμίσεις, όχι κώδικας

Η αξία: **μετρημένη ρύθμιση** στα δεδομένα και το υλικό του πελάτη — δουλειά σε **συμβουλευτικό επίπεδο**.

Το κρατάμε ως έχει; Σενάρια και εναλλακτικές

Κρατάμε το RAGGit, αλλάζουμε μόνο το deployment

- Ίδια εγκατάσταση on-prem, όπως παρουσιάστηκε.
- Private cloud: VM με GPU που ελέγχει ο οργανισμός — ήδη υποστηρίζεται με ρύθμιση.
- GPU VM παρόχοι: Vast.AI, DigitalOcean Paperspace, Azure HPC

Αντικαθιστούμε το RAGGit με έτοιμη λύση

- Ollama ή LM Studio μόνο: τοπικό μοντέλο χωρίς βιβλιοθήκη εγγράφων.
- Azure OpenAI: ισχύς και πολυπλοκότητα, αλλά τα δεδομένα φεύγουν από τον οργανισμό.
- RagFlow (open source) και Gendox: έτοιμες RAG πλατφόρμες.

Η αναζήτηση γίνεται στο εταιρικό περιβάλλον.

Τα έγγραφα και το μοντέλο μένουν στην εγκατάσταση.

Η εταιρεία διαχειρίζεται την πρόσβαση και τα αντίγραφα ασφαλείας.

Ευχαριστώ για την προσοχή!

Ερωτήσεις;

Ανδρέας Δασκαλόπουλος · Patras Tech Talk · 26 Νοεμβρίου 2026

github.com/daskas-welt/raggit